```
  @Autowired
  private OutputDestination output;                          ❸

  @Autowired
  private ObjectMapper objectMapper;                         ❹

  @Test
  void whenOrderAcceptedThenDispatched() throws IOException {
    long orderId = 121;
    Message<OrderAcceptedMessage> inputMessage = MessageBuilder
      .withPayload(new OrderAcceptedMessage(orderId)).build();
    Message<OrderDispatchedMessage> expectedOutputMessage = MessageBuilder
      .withPayload(new OrderDispatchedMessage(orderId)).build();

    this.input.send(inputMessage);                           ❺
    assertThat(objectMapper.readValue(output.receive().getPayload(),
      OrderDispatchedMessage.class))
      .isEqualTo(expectedOutputMessage.getPayload());        ❻
  }
}
```

❶ Configures the test binder

❷ Represents the input binding packlabel-in-0

❸ Represents the output binding packlabel-out-0

❹ Uses Jackson to deserialize JSON message payloads to Java objects

❺ Sends a message to the input channel

❻ Receives and asserts a message from the output channel

WARNING If you use IntelliJ IDEA, you might get a warning that `InputDestination`, `OutputDestination`, and `ObjectMapper` cannot be autowired. Don't worry. It's a false positive. You can get rid of the warning by annotating the field with `@SuppressWarnings("SpringJavaInjectionPointsAutowiringInspection")`.

Message brokers like RabbitMQ deal with binary data, so any data flowing through them is mapped to `byte[]` in Java. The conversion between bytes and DTOs is handled by Spring Cloud Stream transparently. But just like for messages, we need to handle that explicitly in this test scenario when asserting the content of the message received from the output channel.